Support élève (Thème 3)

Objectifs : Thème 3 : Structures linéaires

Dans ce chapitre, nous allons étudier différentes structures de données linéaires.

Nous chercherons notamment à comprendre :

- ce qu'est une structure de données abstraite ;
- comment fonctionne une liste ;
- comment implémenter une liste chaînée ;
- comment fonctionne une file et le principe FIFO ;
- comment fonctionne une pile et le principe LIFO ;
- comment implémenter ces structures à l'aide de classes Python ;
- comment utiliser une pile pour implémenter une file et inversement.

À la fin de ce chapitre, vous devrez être capables de choisir une structure de données adaptée à un problème, de simuler les opérations qui lui sont appliquées et d'implémenter les opérations fondamentales d'une liste chaînée, d'une pile et d'une file en Python.

 Exercice 1 : Ba-liste-ique

1. Voici une série d'instructions. Les instructions ci-dessous s'enchaînent. Expliquez ce qui se passe à chacune des étapes :

```
L = vide()
estVide(L)
ajouteEnTete(3,L)
estVide(L)
ajouteEnTete(5,L)
ajouteEnTete(8,L)
t = supprEnTete(L)
L1 = vide()
L2 = cons(8, cons(5, cons(3, L1)))
```

2. Voici une série d'instructions. Les instructions ci-dessous s'enchaînent. Expliquez ce qui se passe à chacune des étapes :

```
L = vide()
ajoutEnTete(10,L)
ajoutEnTete(9,L)
ajoutEnTete(7,L)
L1 = vide()
L2 = cons(5, cons(4, cons(3, cons(2, cons(1, cons(0,L1))))))
```

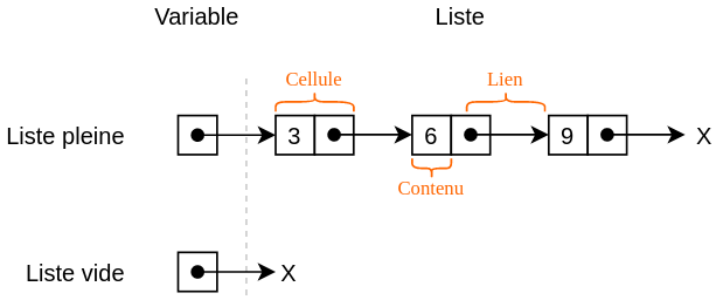
Activité 1 : Implémentation

1. Écrire une classe `Cellule`, possédant deux attributs : `contenu` (la valeur de la cellule) et `suivante` (une instance de la classe `Cellule`).

On pourra alors créer une liste chaînée de la façon suivante :

```
lst = Cellule(3, Cellule(5, Cellule(1, None)))
```

La liste construite sera de la forme suivante :



2. Écrire les lignes de code permettant d'afficher la valeur de toutes les cellules.

Exercice 2 : Ecris liste oire

On dispose de l'interface offrant les fonctionnalités suivantes :

1. `ListeC()` : crée une liste vide ;
2. `est_vide()` : indique si la liste est vide ;
3. `ajoute_tete()` : insère un élément en tête de liste ;
4. `renvoie_tete()` : renvoie la valeur de l'élément en tête de liste et le supprime de la liste.

1. On considère l'enchaînement d'opérations ci-dessous. Écrire à chaque étape l'état de la liste `lst` et la valeur éventuellement renvoyée.

```
lst = ListeC()
lst.ajoute_tete(3)
lst.ajoute_tete(5)
lst.ajoute_tete(1)
lst.renvoie_tete()
lst.est_vide()
lst.ajoute_tete(2)
lst.renvoie_tete()
lst.renvoie_tete()
lst.renvoie_tete()
lst.est_vide()
```

Activité 2 : Classe à Dallas

1. Ajouter une méthode `__str__` à votre classe `Cellule`. Elle affichera le contenu de la cellule et celui des suivantes si elles existent.

   Elle renverra donc : 3 -> 5 -> 1 pour la liste `lst` créée précédemment.

2. Compléter la classe `ListeC` suivante :

```
class ListeC:
    def __init__(self):
        self.tete = None  # Liste vide

    def ajoute_tete(self, valeur):
        '''
        Créer une nouvelle cellule contenant la valeur et l'ajouter en tête.
        '''
        ...
        ...
        ...

    def est_vide(self):
        '''
        Indiquer si la liste est vide ou non.
        @return : (bool) True si vide, False sinon
        '''
        ...
        ...
        ...

    def renvoie_tete(self):
        '''
        Renvoyer le contenu de la tête de la liste et la retirer
        '''
```

```
                ...
                ...
                ...
                ...
                ...
                ...
                ...
                ...
                ...

            def __str__(self):  # affichage de la liste
                return str(self.tete) if not self.est_vide() else ""
```

3. Écrire la suite d'instructions permettant de vérifier votre réponse à l'exercice 2.

```

```



Exercice 3 : File dans ta chambre

Dans les différentes questions, vous considérez pouvoir utiliser les opérations de la SDA donnée ci-dessous.

```
class File:
    def __init__(self):
        ...

    def enfiler(self, x):
        ...

    def defiler(self):
        ...

    def est_vide(self):
        ...
```

1. Écrire une suite d'instructions permettant de créer une instance de la classe `File` affectée à la variable `ma_file` contenant les éléments `1`, `2` et `3` insérés dans cet ordre.

```

```

2. Donner l'affichage obtenu après les appels suivants.

```
# On considère ma_deuxieme_file bien initialisée comme une file vide
ma_deuxieme_file.enfiler(5)
ma_deuxieme_file.enfiler(4)
ma_deuxieme_file.enfiler(3)
ma_deuxieme_file.defiler()
ma_deuxieme_file.defiler()
ma_deuxieme_file.enfiler(4)
ma_deuxieme_file.enfiler(ma_deuxieme_file.defiler())
```

3. Donner une implémentation de l'opération `taille` renvoyant le nombre d'éléments d'une file en utilisant seulement les éléments Python de base et les opérations de la SDA données précédemment.

4. Donner une implémentation de l'opération `inverser` permettant d'inverser les éléments de la file.
La file d'ordre `1`, `2` et `3` deviendra donc la file d'ordre `3`, `2` et `1` après exécution de l'opération `inverser`.